

RAG Foundations

Course Overview

Module	Topic	Key Concepts
RAG Foundations	Context Limitations	Context window constraints, token overflow, cost vs context trade-offs
RAG Foundations	Chunking Strategies	Fixed, semantic, recursive chunking; overlap strategies
RAG Foundations	Context Optimization	Chunk size tuning, overlap tuning, retrieval impact

RAG FOUNDATIONS › TOPIC 1

Context Limitations

Understanding the inherent constraints of LLM context windows and how they motivate RAG architecture.

1.1 What Is a Context Window?

Every Large Language Model (LLM) processes text within a fixed-size **context window** — the maximum amount of text (measured in tokens) the model can 'see' in a single inference call. This window includes the system prompt, conversation history, retrieved documents, and the model's own output. Once the input exceeds this limit, older tokens are silently dropped or the API returns an error, resulting in lost context, degraded accuracy, or failed requests.

Common LLM Context Window Sizes

Model	Context Window	Approx. Pages	RAG Suitability
GPT-3.5 Turbo	4K – 16K tokens	~3 – 12 pages	Limited — small docs only
GPT-4 / GPT-4o	8K – 128K tokens	~6 – 100 pages	Good for mid-size corpora
Claude 3.5 Sonnet	200K tokens	~150 pages	Excellent; large doc support
Gemini 1.5 Pro	1M tokens	~750 pages	Very large; still benefits from RAG
LLaMA 3 (local)	8K – 128K tokens	~6 – 100 pages	RAG strongly recommended

1.2 Token Overflow — What Happens When Context is Exceeded?

When input text exceeds the model's context window, one of three failure modes occurs. Understanding these modes is essential for designing robust RAG pipelines.

Hard Truncation

- API raises a token-limit error
- Request fails entirely
- Application must handle the exception
- Common in: OpenAI, Anthropic APIs without streaming

Silent Truncation

- Oldest tokens silently dropped (left truncation)
- Model never 'sees' the beginning of the document
- Answers appear plausible but lack critical context
- Hardest failure mode to detect

Context Dilution

- Large window ≠ effective reasoning over the whole window
- LLMs suffer 'lost in the middle' degradation
- Relevant info buried at position 40-60% is frequently ignored
- RAG retrieves only the most relevant top-k chunks

1.3 Cost vs. Context Trade-offs

Larger context windows come with direct API cost implications. Most commercial LLM providers charge per input token, meaning that naively stuffing entire documents into prompts leads to exponentially growing costs. RAG solves this by retrieving only the most relevant chunks.

Approach	Tokens per Query	Cost Impact	Accuracy Risk
Full-doc in prompt	50K – 200K+	Very High (\$\$\$)	High (dilution)
Naive chunking (RAG)	1K – 4K	Low (\$)	Medium (retrieval miss)
Optimised RAG	2K – 6K	Low–Medium (\$\$)	Low
Hybrid (RAG + re-rank)	2K – 8K	Medium (\$\$)	Very Low

Key Insight: Why RAG?

- RAG decouples knowledge storage (vector DB) from reasoning (LLM).
- Only 2-8 retrieved chunks are sent to the LLM per query — not the entire corpus.
- This keeps token usage predictable, cost low, and accuracy high.
- The LLM focuses on reasoning, not memorisation.

Chunking Strategies

Techniques for splitting documents into retrievable units that balance semantic coherence with retrieval precision.

Chunking is the process of splitting large documents into smaller segments that are independently embedded and stored in a vector database. The chunking strategy directly determines retrieval quality — poorly chunked documents yield irrelevant or incomplete context regardless of how good the embedding model is.

2.1 Fixed-Size Chunking

The simplest strategy: split text into chunks of exactly N tokens (or characters), optionally with an overlap of M tokens between adjacent chunks.

Attribute	Detail
Split Criterion	Every N tokens / characters regardless of content
Overlap	Optional M-token overlap between consecutive chunks
Implementation	LangChain CharacterTextSplitter / RecursiveCharacterTextSplitter
Best For	Homogeneous text (logs, transcripts, single-topic docs)
Weakness	Splits mid-sentence or mid-paragraph, breaking semantic units
Typical Chunk Size	256 – 512 tokens for dense text; 128 – 256 for Q&A; style

```
Code sketch (LangChain): text_splitter = CharacterTextSplitter(chunk_size=512,
chunk_overlap=50) | docs = text_splitter.split_documents(raw_docs)
```

2.2 Semantic Chunking

Instead of splitting at fixed intervals, semantic chunking splits at natural semantic boundaries — sentence groups, paragraphs, or topic shifts. The most advanced form uses embedding similarity between adjacent sentences to detect topic transitions.

Step 1 — Sentence Segmentation

Tokenise document into individual sentences using spaCy, NLTK, or regex.

Step 2 — Embedding Sentences

Generate vector embeddings for each sentence using a bi-encoder model (e.g. all-MiniLM-L6-v2).

Step 3 — Similarity Scoring

Calculate cosine similarity between consecutive sentence pairs.

Step 4 — Boundary Detection

When similarity drops below threshold θ (e.g. 0.65), mark a chunk boundary.

Step 5 — Chunk Assembly

Merge sentences within a boundary into a single chunk; apply min/max token constraints.

Advantages	Limitations
Preserves semantic coherence within each chunk	Slower — requires sentence-level embedding pass
Variable-size chunks match natural information density	Sensitive to threshold θ — needs tuning per domain
Better recall for factual Q&A; and summarisation tasks	Higher upfront compute cost during indexing
Reduces split-sentence context loss	Overkill for short, homogeneous documents

2.3 Recursive Chunking

LangChain's **RecursiveCharacterTextSplitter** is the most widely-used production chunking strategy. It applies a hierarchy of separators — paragraphs → sentences → words → characters — recursively until chunks are within the target size. This preserves structural boundaries while guaranteeing a size constraint.

Priority	Separator	When Used
1 (highest)	Double newline (<code>\n\n</code>)	Paragraph boundaries — preferred split point
2	Single newline (<code>\n</code>)	Line breaks within a paragraph
3	Period + space (<code>. </code>)	Sentence boundaries
4	Space (<code> </code>)	Word boundaries
5 (fallback)	Empty string (character)	Hard character-level split — last resort

```
Code sketch (LangChain): splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50, separators=["\n\n", "\n", ". ", " ", ""]) | chunks =
splitter.split_text(doc)
```

2.4 Overlap Strategies

Chunk overlap deliberately repeats M tokens at the end of one chunk and the start of the next. This guards against context truncation at boundaries — ensuring that information spanning the split point appears in at least one complete chunk.

Overlap Setting	Effect	Recommended When
0 (no overlap)	Hard boundary — context may be cut	Structured docs with clear delimiters (JSON, Markdown headings)
5 – 10%	Light continuity across boundaries	General prose, product documentation
15 – 20%	Strong cross-boundary context	Legal, medical, research papers with cross-referencing
> 25%	High redundancy; more storage/compute	Only when missing context is catastrophic (e.g. compliance docs)

2.5 Strategy Selection Guide

Strategy	Complexity	Chunk Quality	Best Use Case
Fixed-size	Low	Medium	Logs, uniform text, fast prototyping
Semantic	High	Very High	Q&A;, summarisation, research corpora
Recursive	Medium	High	General-purpose production RAG
Sentence-window	Medium	High	Conversational agents, chatbots
Document-aware	High	Very High	PDFs with headings/tables/figures

Context Optimization

Systematic techniques to tune chunk size, overlap, and retrieval parameters for maximum accuracy.

Context optimisation is the iterative engineering process of adjusting RAG pipeline parameters to maximise retrieval quality and answer accuracy. The three primary levers are chunk size, chunk overlap, and retrieval top-k. Changes to any one parameter have cascading effects on the others.

3.1 Chunk Size Tuning

Chunk size is the single most impactful RAG parameter. It controls the granularity of indexed units and directly affects both retrieval precision and the completeness of context sent to the LLM.

Chunk Size	Retrieval Precision	Context Completeness	Recommended Task
64 – 128 tokens	Very High	Low (fragments)	Keyword/fact lookup, entity extraction
128 – 256 tokens	High	Medium	Short Q&A;, FAQ bots
256 – 512 tokens	Medium-High	Good	General Q&A;, knowledge bases (default)
512 – 1024 tokens	Medium	High	Summarisation, reasoning tasks
1024+ tokens	Low	Very High	Document-level summarisation

Small vs Large Chunk Trade-off

- Small chunks → higher precision but may miss surrounding context.
- Large chunks → richer context but embed multiple topics, hurting cosine similarity.
- The 'sweet spot' is domain-specific: legal text favours larger chunks; FAQs favour smaller.
- Use RAGAS (Retrieval Augmented Generation Assessment) metrics to measure empirically.

3.2 Overlap Tuning

Overlap tuning is secondary to chunk-size tuning but essential for eliminating boundary-split failures. The goal is to find the minimum overlap that prevents semantic breaks without inflating index size or embedding costs.

Start value	Set overlap = 10% of chunk_size as the baseline.
Increase when	Queries frequently return incomplete answers that 'feel cut off' mid-thought.
Decrease when	Index is too large or you observe duplicate content in retrieved chunks.

Measure with	Context Recall metric in RAGAS; also manual inspection of boundary chunks.
Hard limit	Never exceed 50% overlap — diminishing returns and embedding noise dominate.

3.3 Retrieval Impact — Top-K and Re-ranking

Beyond chunking, retrieval configuration (top-k, similarity threshold, re-ranking) is equally critical for context quality. Retrieving too few chunks causes missed context; too many dilutes the signal and increases token costs.

Parameter	Typical Value	Effect on Quality	Effect on Cost
top-k (retrieval)	3 – 5	Core recall; lower k = higher precision, lower recall	Linear — fewer chunks = fewer tokens
Similarity threshold	0.70 – 0.85	Filters irrelevant chunks; high threshold = strict matching	Reduces tokens; risk of no results
Re-ranker (cross-encoder)	Top 20 → re-rank → top 5	Dramatically improves precision with minimal recall loss	Adds re-ranker inference cost (~10–20ms/query)
MMR (Max Marginal Relevance)	k=5, λ=0.7	Balances relevance with diversity; reduces redundancy	Minimal overhead vs. standard top-k
Hybrid search (dense+sparse)	Weighted combination	Best of semantic and keyword matching; strong recall	Slightly higher index storage

3.4 Optimisation Workflow

Phase 1 — Baseline

Set chunk_size=512, chunk_overlap=50, top_k=4

Run RAGAS evaluation on 50-100 representative queries

Record: Context Precision, Context Recall, Answer Faithfulness

Phase 2 — Chunk Size Grid Search

Test: [128, 256, 512, 768, 1024] token sizes

Hold overlap at 10% of chunk size

Plot Context Recall vs Chunk Size — find the 'knee' of the curve

Phase 3 — Overlap Tuning

Fix chunk_size at best value from Phase 2

Test: [0%, 5%, 10%, 15%, 20%] overlap

Optimise for context recall without index size blowup

Phase 4 — Retrieval Tuning

Tune top_k (3, 5, 7, 10) and similarity threshold

Add re-ranker (BGE-Reranker, Cohere Rerank) and measure delta

Consider hybrid search if keyword recall is important

Phase 5 — Production Monitoring

Log per-query chunk usage, retrieval latency, and answer confidence

Set up drift detection — re-tune as document corpus grows

Automate re-indexing pipeline with GitHub Actions or Airflow

3.5 Evaluation Metrics (RAGAS Framework)

RAGAS provides a reference-free evaluation framework for RAG pipelines, enabling systematic measurement of both retrieval and generation quality.

Metric	Measures	Target	Impacted By
Context Precision	Are retrieved chunks relevant?	> 0.80	top-k, similarity threshold, embedding model
Context Recall	Is all necessary info retrieved?	> 0.75	Chunk size, overlap, top-k
Answer Faithfulness	Is the answer grounded in the chunks?	> 0.85	Prompt design, LLM selection
Answer Relevance	Does the answer address the question?	> 0.80	Retrieval quality, LLM capability
Context Entity Recall	Are named entities in ground truth retrieved?	> 0.70	Chunking granularity, hybrid search

Section Summary — Context Optimization

1. Chunk size is the primary lever: 256-512 tokens is a safe default starting point.
2. Set overlap at 10% of chunk size; increase to 15-20% for domain-specific accuracy needs.
3. Retrieve top-k=4 to 5 by default; add a cross-encoder re-ranker for high-stakes apps.
4. Always evaluate with RAGAS before and after any parameter change.
5. Monitor production metrics — optimal parameters shift as document corpus evolves.